Eng 1.3. Introduction to Cryptography

Task 1 - Introduction

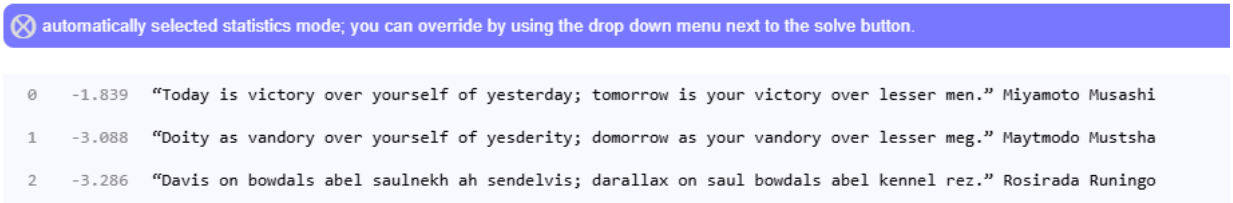
The purpose of this room is to introduce users to basic cryptography concepts such as:

Symmetric encryption, such as AES
Asymmetric encryption, such as RSA
Diffie-Hellman Key Exchange
Hashing
PKI

Caesar cipher is considered a substitution cipher because each letter in the alphabet is substituted with another. Another type of cipher is called transposition cipher, which encrypts the message by changing the order of the letters. We don't really need to use the encryption key to decrypt the received ciphertext, "Uyv sxd gyi siqvw x sinduxjd pvzjdw po axffojdz xgxo wsxcc wuidvw." As shown in the figure below, using a website such as quipqiup(opens in new tab), it will take a moment to discover that the original text was "The man who moves a mountain begins by carrying away small stones." This example clearly indicates that this algorithm is broken and should not be used for confidential communication.

> Q. You have received the following encrypted message: "Xjnvw lc sluxjmw jsqm wjpmcqbg jg wqcxqmnvw; xjzjmmjd lc wjpm sluxjmw jsqm bqccqm zqy." Zlwvzjxj Zpcvcol. You can guess that it is a quote. Who said it?

Miyamoto Musashi

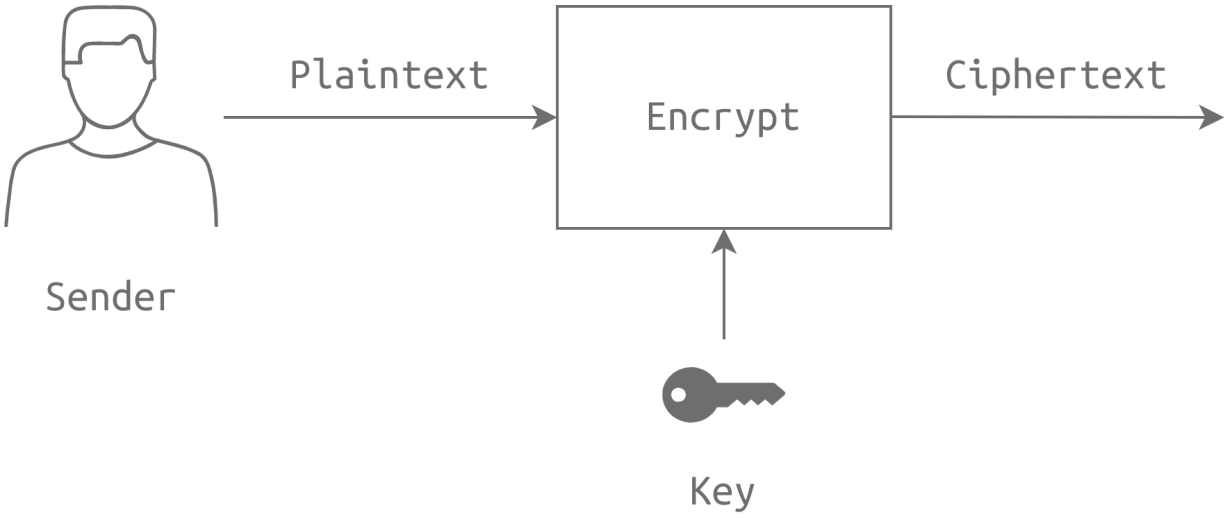


Task 2 - Symmetric Encryption

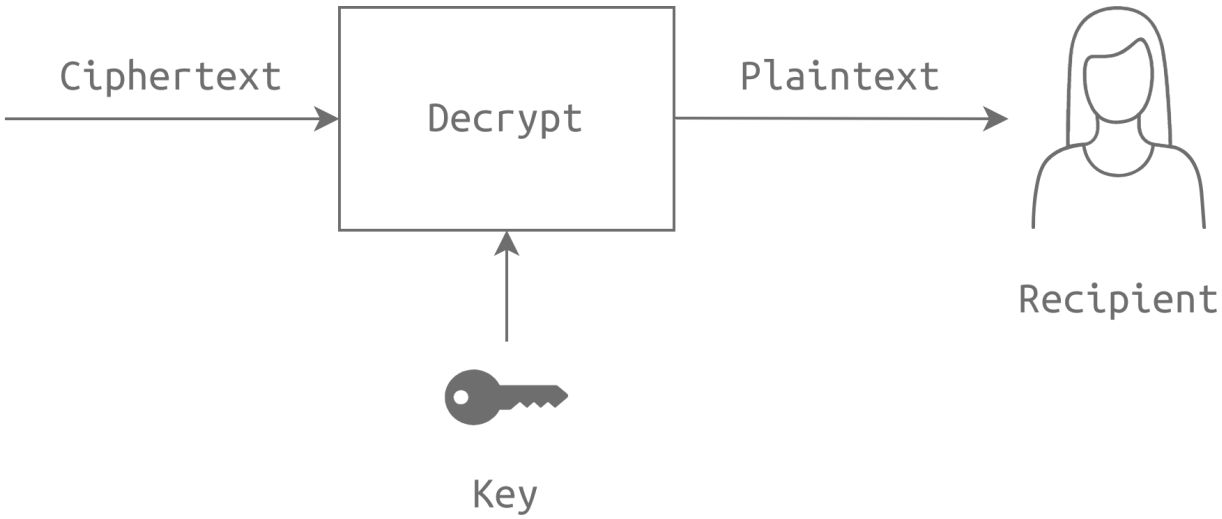
Cryptographic Algorithm or Cipher: This algorithm defines the encryption and decryption processes.
Key: The cryptographic algorithm needs a key to convert the plaintext into ciphertext and vice versa.
plaintext is the original message that we want to encrypt.
ciphertext is the message in its encrypted form.

Encryption Algorithm	Notes
AES, AES192, and AES256	AES with a key size of 128, 192, and 256 bits
IDEA	International Data Encryption Algorithm (IDEA)
3DES	Triple DES (Data Encryption Standard) and is based on DES. We should note that 3DES will be deprecated in 2023 and disallowed in 2024.
CAST5	Also known as CAST-128. Some sources state that CAST stands for the names of its authors: Carlisle Adams and Stafford Tavares.
BLOWFISH	Designed by Bruce Schneier
TWOFISH	Designed by Bruce Schneier and derived from Blowfish
CAMELLIA128, CAMELLIA192, and CAMELLIA256	Designed by Mitsubishi Electric and NTT in Japan. Its name is derived from the flower camellia japonica.

Symmetric encryption algorithm uses the same key for encryption and decryption. Consequently, the communicating parties need to agree on a secret key before being able to exchange any messages. In the following figure, the sender provides the *encrypt* process with the plaintext and the key to get the ciphertext. The ciphertext is usually sent over some communication channel.



On the other end, the recipient provides the *decrypt* process with the same key used by the sender to recover the original plaintext from the received ciphertext. Without knowledge of the key, the recipient won't be able to recover the plaintext.



GNU Privacy Guard: The GNU Privacy Guard(opens in new tab), also known as GnuPG or GPG, implements the OpenPGP standard. We can encrypt a file using GnuPG (GPG) using the following command:

`gpg --symmetric --cipher-algo CIPHER message.txt`, where CIPHER is the name of the encryption algorithm.

You can decrypt using the following command:

```
gpg --output original_message.txt --decrypt message.gpg
```

OpenSSL Project: The OpenSSL Project(opens in new tab) maintains the OpenSSL software.

We can encrypt a file using OpenSSL using the following command:

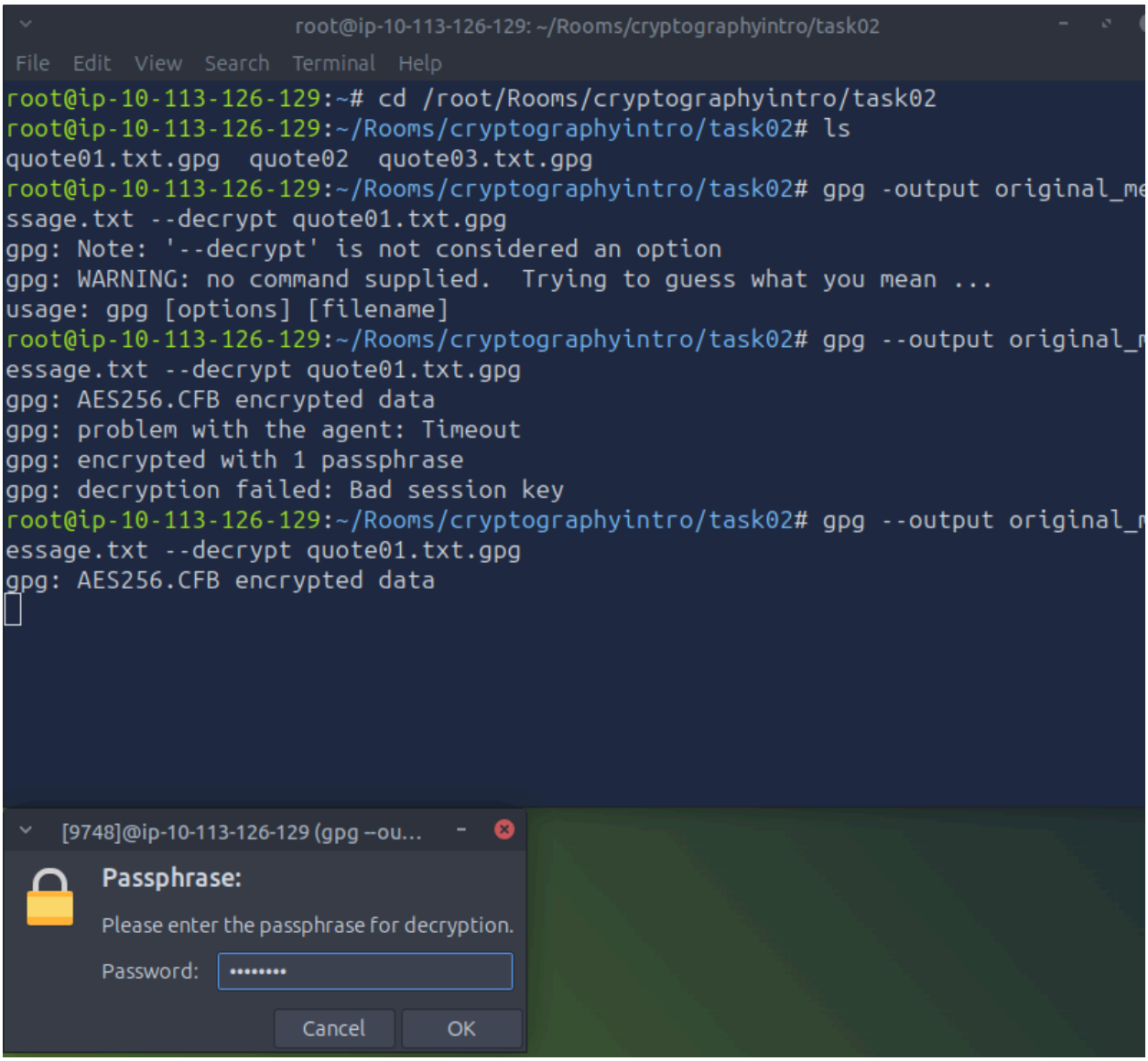
```
openssl aes-256-cbc -e -in message.txt -out encrypted_message
```

We can decrypt the resulting file using the following command:

```
openssl aes-256-cbc -d -in encrypted_message -out original_message.txt
```

> Q.1. Decrypt the file quote01 encrypted (using AES256) with the key s!kR3T55 using gpg. What is the third word in the file?

waste



> Q.2. Decrypt the file quote02 encrypted (using AES256-CBC) with the key s!kR3T55 using openssl. What is the third word in the file?

science

```
root@ip-10-113-126-129:~/Rooms/cryptographyintro/task02# openssl aes-256-cbc -d
-in quote02 -out original_message.txt
enter AES-256-CBC decryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
root@ip-10-113-126-129:~/Rooms/cryptographyintro/task02# cat mymessage.txt
cat: mymessage.txt: No such file or directory
root@ip-10-113-126-129:~/Rooms/cryptographyintro/task02# cat original_message.tx
t
The true science of martial arts means practicing them in such a way that they w
ill be useful at any time, and to teach them in such a way that they will be use
ful in all things.
Miyamoto Musashi
root@ip-10-113-126-129:~/Rooms/cryptographyintro/task02#
```

> Q.3. Decrypt the file quote03 encrypted (using CAMELLIA256) with the key s!kR3T55 using gpg. What is the third word in the file?

understand

```
root@ip-10-113-126-129:~/Rooms/cryptographyintro/task02# ls
original_message.txt  quote01.txt.gpg  quote02  quote03.txt.gpg
root@ip-10-113-126-129:~/Rooms/cryptographyintro/task02# gpg --output m3.txt --d
ecrypt quote03.txt.gpg
gpg: CAMELLIA256.CFB encrypted data
gpg: encrypted with 1 passphrase
root@ip-10-113-126-129:~/Rooms/cryptographyintro/task02# cat m3.txt
You must understand that there is more than one path to the top of the mountain.
Miyamoto Musashi
root@ip-10-113-126-129:~/Rooms/cryptographyintro/task02#
```

Task 3 - Asymmetric Encryption

Asymmetric encryption makes it possible to exchange encrypted messages without a secure channel; we just need a reliable channel. By reliable channel, we mean that we are mainly concerned with the channel's integrity and not confidentiality. When using an asymmetric encryption algorithm, we would generate a key pair: a public key and a private key. The public key is shared with the world, or more specifically, with the people who want to communicate with us securely. The private key must be saved securely, and we must never let anyone access it. Moreover, it is not feasible to derive the private key despite the knowledge of the public key.

Three RSA commands:

- `openssl genrsa -out private-key.pem 2048`: With `openssl`, we used `genrsa` to generate an RSA private key. Using `out`, we specified that the resulting private key is saved as `private-key.pem`. We added `2048` to specify a key size of 2048 bits.
- `openssl rsa -in private-key.pem -pubout -out public-key.pem`: Using `openssl`, we specified that we are using the RSA algorithm with the `rsa` option. We specified that we wanted to get the public key using `pubout`. Finally, we set the private key as input using `in private-key.pem` and saved the output using `out public-key.pem`.
- `openssl rsa -in private-key.pem -text -noout`: We are curious to see real RSA variables, so we used `text -noout`. The values of *p*, *q*, *N*, *e*, and *d* are `prime1`, `prime2`, `modulus`, `publicExponent`, and `privateExponent`, respectively.

If we already have the recipient's public key, we can encrypt it with the command `openssl pkeyutl -encrypt -in plaintext.txt -out ciphertext -inkey public-key.pem -pubin`

The recipient can decrypt it using the command `openssl pkeyutl -decrypt -in ciphertext -inkey private-key.pem -out decrypted.txt`

```
root@ip-10-113-126-129:~/Rooms/cryptographyintro/task02# cd ../task03
root@ip-10-113-126-129:~/Rooms/cryptographyintro/task03# ls
ciphertext_message   public-key-alice.pem
private-key-bob.pem  public-key-bob.pem
root@ip-10-113-126-129:~/Rooms/cryptographyintro/task03# openssl pkeyutl -decryp
t -in ciphertext_message -inkey private-key-bob.pem -out decrypted.txt
root@ip-10-113-126-129:~/Rooms/cryptographyintro/task03# cat decrypted.txt
"Perception is strong and sight weak. In strategy it is important to see distant
 things as if they were close and to take a distanced view of close things."
Miyamoto Musashi
```

> Q1. Take a look at Bob's private RSA key. What is the last byte of *p*?

p=prime1:e7

```
root@ip-10-113-126-129:~/Rooms/cryptographyintro/task03# openssl rsa -in private
-key-bob.pem -text -noout
Private-Key: (2048 bit, 2 primes)
modulus:
    00:e8:5e:73:7d:54:55:0a:cc:56:64:87:b3:4b:8e:
    24:df:96:b8:b9:5f:19:d4:71:a5:b9:5a:a8:d9:ab:
    b4:7b:7f:59:08:c5:9c:47:0d:73:92:97:b8:ef:67:
    b7:a6:5a:59:2c:e3:4c:ca:7f:53:1c:9e:34:32:0e:
    c6:7c:60:b2:d6:1f:30:b2:ed:da:14:e9:15:78:80:
    71:92:3c:26:32:a9:2b:3a:15:4e:48:2d:93:04:a5:
    21:c7:da:15:6c:dd:bc:89:0e:cc:54:be:84:d6:40:
    b8:47:59:d1:b2:27:c9:0d:43:55:de:33:dd:01:8f:
    bf:6c:3e:79:31:dd:e4:90:8d:c3:35:72:31:85:15:
    ae:ac:5a:96:c8:34:90:0e:32:4e:86:45:55:78:fb:
    13:ed:a4:fb:f0:64:b4:61:04:f6:7c:e3:56:aa:03:
    a3:43:1e:40:0b:98:1f:73:66:4a:5c:3a:25:69:2c:
    d9:92:f8:69:c1:5b:61:b7:f2:3a:68:28:e9:2b:75:
    08:9c:a4:63:9e:71:2b:63:aa:99:75:cf:78:00:23:
    fc:5a:df:2d:95:14:2f:e6:10:5d:a0:ff:4d:07:c8:
    d3:bb:2d:8f:0d:8a:fc:ab:43:5d:35:53:dc:72:a2:
    74:5b:c0:88:0d:ee:c3:1f:7b:1c:74:1a:5e:e1:c1:
    88:31
```

```
prime1:
    00:ff:ea:65:3e:e5:96:96:0b:66:55:f1:f9:d0:37:
    66:e9:35:a5:c3:43:ca:66:75:40:49:46:8d:85:a7:
    ff:f4:73:97:69:11:a1:1e:37:f9:e3:38:cb:c0:5e:
    56:e9:1a:0d:f2:9f:80:56:87:2a:99:bb:88:8e:93:
    35:5a:9a:c6:f7:99:44:90:88:09:33:a6:0d:ea:b4:
    56:98:66:20:9c:34:e7:b9:33:64:4f:08:01:08:62:
    44:68:8f:df:79:0d:84:2b:77:e7:03:8b:3c:7a:e3:
    e0:e0:ee:23:64:22:51:ed:dd:b8:1c:b3:75:c4:3f:
    4a:cf:fc:7c:57:0b:95:75:e7
```

> Q.2. Take a look at Bob's private RSA key. What is the last byte of *q*?

q=prime2=27

```
prime2:
    00:e8:72:11:5c:b5:5c:14:19:85:ce:e7:d2:e9:54:
    7b:58:ae:32:e9:e6:39:a7:65:b4:90:2f:53:b5:9d:
    22:62:84:fe:52:86:f5:01:a2:9c:b0:4f:80:ee:d4:
    07:27:3b:69:02:70:33:da:7d:97:56:b9:3e:f3:a1:
    84:9e:73:6a:47:e5:99:8c:44:86:75:c1:bf:71:89:
    06:b0:ee:dd:16:45:e7:05:fa:02:bd:e6:3e:b7:f2:
    fe:e7:22:0b:ed:ca:23:a0:68:0b:fe:fb:c3:57:19:
    21:58:6e:73:1d:9d:3c:2a:8a:c1:7e:ea:73:67:5a:
    cb:3d:a8:9b:be:50:08:9e:27
```

Task 4 - Diffie-Hellman Key Exchange

Let's take a look at actual Diffie-Hellman parameters. We can use `openssl` to generate them; we need to specify the option `dhparam` to indicate that we want to generate Diffie-Hellman parameters along with the specified size in bits, such as `2048` or `4096`. In the console output below, we can view the prime number `P` and the generator `G` using the command `openssl dhparam -in dhparams.pem -text -noout`. (This is similar to what we did with the RSA private key.)

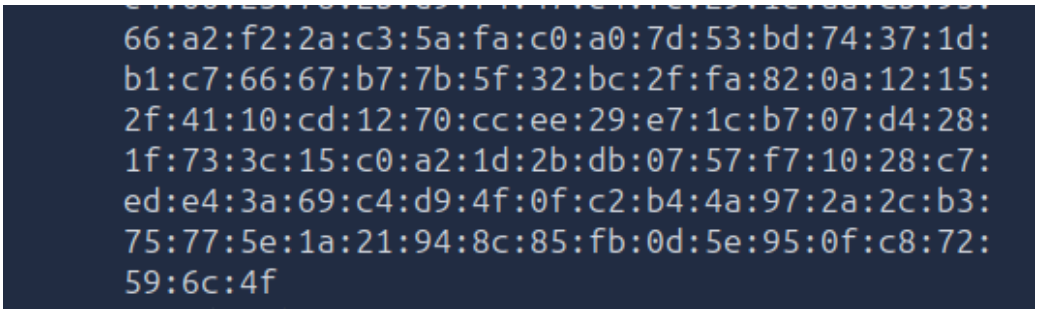
> Q.1. On the AttackBox, you can find the directory for this task located at /root/Rooms/cryptographyintro/task04; alternatively, you can use the task file from Task 2 to work on your own machine. A set of Diffie-Hellman parameters can be found in the file dhparam.pem. What is the size of the prime number in bits?

4096

```
root@ip-10-113-126-129:~/Rooms/cryptographyintro/task04# openssl dhparam -in dhp
arams.pem -text -noout
    DH Parameters: (4096 bit)
    P:
        00:c0:10:65:c6:ad:ed:88:04:88:1e:e7:50:1b:30:
        0f:05:2c:2d:d4:ea:60:44:9e:2a:f7:90:02:89:a4:
        7e:05:99:32:38:dc:75:50:0a:c7:f6:6b:f7:b4:9a:
        df:ef:ca:e0:ce:55:5d:31:48:3e:9c:35:5a:ad:03:
        9c:87:d7:1c:48:e4:2e:29:dc:a3:90:81:23:7f:fa:
        30:5c:fb:d8:62:7b:96:35:ef:9a:0f:84:49:c4:48:
        97:b5:63:38:91:01:49:f1:42:15:fd:da:84:a6:90:
```

> Q.2. What is the prime number's last byte (least significant byte)?

4f



Task 5 - Hashing

A cryptographic hash function is an algorithm that takes data of arbitrary size as its input and returns a fixed size value, called *message digest* or *checksum*, as its output. For example, `sha256sum` calculates the SHA256 (Secure Hash Algorithm 256) message digest.

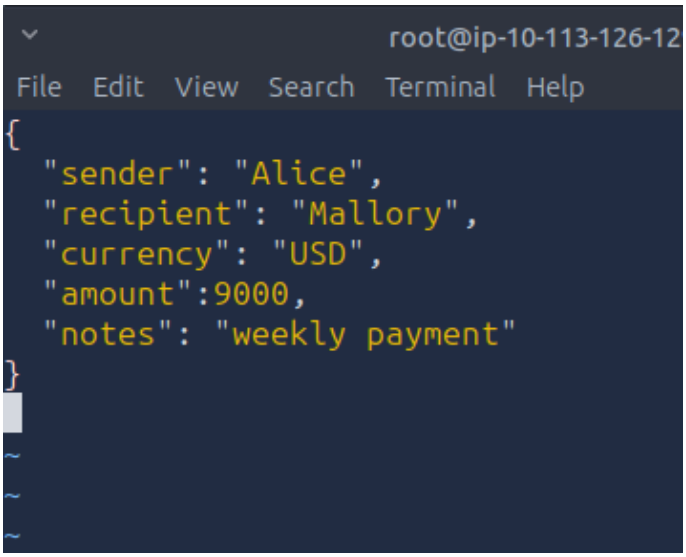
HMAC: Hash-based message authentication code (HMAC) is a message authentication code (MAC) that uses a cryptographic key in addition to a hash function.To calculate the HMAC on a Linux system, you can use any of the available tools such as `hmac256` (or `sha224hmac`, `sha256hmac`, `sha384hmac`, and `sha512hmac`, where the secret key is added after the option `--key`). Below we show an example of calculating the HMAC using `hmac256` and `sha256hmac` with two different key

> Q.1. On the AttackBox, you can find the directory for this task located at /root/Rooms/cryptographyintro/task05; alternatively, you can use the task file from Task 2 to work on your own machine.What is the SHA256 checksum of the file order.json?

2c34b68669427d15f76a1c06ab941e3e6038dacdfb9209455c87519a3ef2c660

> Q.2.Open the file order.json and change the amount from 1000 to 9000. What is the new SHA256 checksum?

11faeec5edc2a2bad82ab116bbe4df0f4bc6edd96adac7150bb4e6364a238466



> Q.3. Using SHA256 and the key 3RfDFz82, what is the HMAC of order.txt?

c7e4de386a09ef970300243a70a444ee2a4ca62413aeaeb7097d43d2c5fac89f

hmac256 3RfDFz82 order.txt

Task 6 - PKI and SSL/TLS

PKI is the infrastructure that manages digital certificates and public/private keys, while SSL/TLS is the protocol that uses those certificates to create secure encrypted communication between systems. PKI establishes trust, and TLS uses that trust to secure data in transit. For a certificate to get signed by a certificate authority, we need to:

1. Generate Certificate Signing Request (CSR): You create a certificate and send your public key to be signed by a third party.
2. Send your CSR to a Certificate Authority (CA): The purpose is for the CA to sign your certificate. The alternative and usually insecure solution would be to self-sign your certificate.

Core Components of PKI
Public Key → Shared openly and used for encryption or signature verification.
Private Key → Kept secret and used for decryption or digital signing.
Digital Certificate → Binds a public key to an identity (like a website or organization).

Certificate Authority (CA) → Trusted organization that issues and signs certificates.
Registration Authority (RA) → Verifies identity before certificate issuance.
Certificate Revocation List (CRL) / OCSP → Used to check if a certificate is revoked.

How PKI Works
A server generates a public/private key pair.
The public key is sent to a CA in a Certificate Signing Request (CSR).
The CA verifies identity and issues a digital certificate.
Clients trust the CA, so they trust the server certificate.
Secure encrypted communication can now happen.

Uses of PKI
HTTPS websites
VPN authentication
Email encryption/signing
Digital signatures
Secure APIs

SSL/TLS: SSL (Secure Sockets Layer) and TLS (Transport Layer Security) are protocols used to secure communication over a network. TLS is the modern, secure replacement for SSL.

Main Goals
Confidentiality → Data is encrypted.
Integrity → Data is not modified in transit.
Authentication → Confirms the server identity.

TLS Handshake Process
Client connects to server.
Server sends its digital certificate.
Client verifies the certificate using trusted CA certificates.
Client and server agree on encryption algorithms and exchange session keys.
A secure encrypted session is established.

Relationship Between PKI and TLS
TLS uses PKI for authentication.
PKI provides certificates and trust validation.
TLS uses those certificates to establish secure encrypted communication.

> Q.1. On the AttackBox, you can find the directory for this task located at `/root/Rooms/cryptographyintro/task06`; alternatively, you can use the task file from Task 2 to work on your own machine. What is the size of the public key in bits?

> Q.2. Till which year is this certificate valid?

openssl x509 -in cert.pem -text | less

```
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            2b:29:0c:2f:b0:52:3a:79:89:1f:82:11:07:bd:9d:84:2a:23:d5:1c
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C = UK, ST = London, L = London, O = Default Company Ltd
        Validity
            Not Before: Aug 11 11:34:19 2022 GMT
            Not After : Feb 25 11:34:19 2039 GMT
        Subject: C = UK, ST = London, L = London, O = Default Company Ltd
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (4096 bit)
```

Task 7 - Authenticating with Passwords

With PKI and SSL/TLS, we can communicate with any server and provide our login credentials while ensuring that no one can read our passwords as they move across the network. This is an example of protecting data in transit. Let's explore how we can safeguard passwords as they are saved in a database, i.e., data at rest.

You were auditing a system when you discovered that the MD5 hash of the admin password is 3fc0a7acf087f549ac2b266baf94b8b1. What is the original password? qwerty123

Use an online MD5 crack tool such as https://www.md5online.org/md5-decrypt.html or https://md5decrypt.net/en/

Task 8 - Cryptography and Data - Example

Cryptography's role starts with checking the certificate. For a certificate to be considered valid, it means it is signed. Signing means that a hash of the certificate is encrypted with the private key of a trusted third party; the encrypted hash is appended to the certificate.

If the third party is trusted, the client will use the third party's public key to decrypt the encrypted hash and compare it with the certificate's hash. However, if the third party is not recognized, the connection will not proceed automatically.

Once the client confirms that the certificate is valid, an SSL/TLS handshake is started. This handshake allows the client and the server to agree on the secret key and the symmetric encryption algorithm, among other things. From this point onward, all the related session communication will be encrypted using symmetric encryption.

The final step would be to provide login credentials. The client uses the encrypted SSL/TLS session to send them to the server. The server receives the username and password and needs to confirm that they match.